

Doc. no.: P0996R1
Date: 2018-03-16
Project: Programming Language C++
Audience: Library Evolution Working Group
Library Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Rebase Library Fundamentals TS on C++17

Table of Contents

- Revision History
 - Revision 0
 - Revision 1
- 1 Introduction
- 2 Stating the problem
- 3 Propose Solution
- 4 Other Directions
- 5 Formal Wording
 - 5.1 Strike out sections that landed in C++17
 - 5.2 Revise The Front Matter
 - 5.3 Global Search and Replace
 - 5.4 Fix Cross-references
- 6 Acknowledgements
- 7 References

Revision History

Revision 0

Original version of the paper for the 2018 Jacksonville meeting.

Revision 1

Simplify the paper to a simple set of editorial suggestions.

1 Introduction

The Library Fundamentals series of TSes are expected to produce new revisions several times per standard cycle. The current draft should be updated to reflect that C++17 was published shortly after the Albuquerque 2017 meeting.

2 Stating the problem

The library Fundamentals TS is expected to serve as a long-running sequence of TSes that provide experience on new library features that may be added to future standards. With the publication of the latest C++ Standard in 2017, we should rebase the current TS on that standard before adding new experimental components. The first of those components are expected to land soon.

3 Propose Solution

This paper proposes the simplest imagined rebasing of the document. It does not attempt to apply new C++17 language features to existing components. For example, it has not performed a review for class template deduction guides. It is expected such reviews will be much simpler once the text for the landed components is excised.

Similarly, this paper makes no attempt to resolve awkward wording updates where the underlying text of the referenced C++ standard has changes substantially between C++14 and C++17. In such cases, it provides a simple issues list to track the necessary updates, which should be provided by experts in the affected components.

First, we propose excising all components that have merged into the main standard, and update any remaining internal cross-references to point to the C++17 standard for their specification.

Then we update document references in clause 1, then update all numbered cross-references into the new standard to use the updated standard ISO clause numbering.

Finally, we give the project editor guidance on how to apply a few simple patterns to change the remaining text to refer to the updated experimental namespace. Similarly, we leave it as an exercise for the project editor to fix up cross-references from the C++14 standard to the C++17 standard.

4 Other Directions

A more detailed rebasing was attempted, but produced a much longer document than the Library Working Group would have an easy time reviewing during a meeting. The majority of the extra text were seen as minor changes performing obvious tasks such as fixing up cross references, and applying consistent editing patterns such as renaming the experimental namespace. It was seen as more appropriate to give editorial direction to the project editor to handle those cases, than have a detailed line-by-line review in LWG session.

5 Formal Wording

5.1 Strike out sections that landed in C++17

Completely excise from the document all the sections marked as **deleted** in the index table below.

- 1 [General](#)
 - 1.1 [Scope](#)
 - 1.2 [Normative references](#)
 - 1.3 [Namespaces, headers, and modifications to standard classes](#)
 - 1.4 [Terms and definitions](#)
 - 1.4.1 [direct non-list initialization](#)
 - 1.5 [Future plans \(Informative\)](#)
 - 1.6 [Feature-testing recommendations \(Informative\)](#)
- 2 [Modifications to the C++ Standard Library](#)
 - 2.1 [Uses-allocator construction](#)
- 3 [General utilities library](#)
 - 3.1 [Utility components](#)
 - 3.1.1 [Header <experimental/utility> synopsis](#)
 - 3.1.2 [Class `erased\_type`](#)
 - 3.2 [Tuples](#)
 - 3.2.1 [Header <experimental/tuple> synopsis](#)
 - 3.2.2 [Calling a function with a tuple of arguments](#)
 - 3.3 [Metaprogramming and type traits](#)
 - 3.3.1 [Header <experimental/type_traits> synopsis](#)
 - 3.3.2 [Other type transformations](#)
 - 3.3.3 [Logical operator traits](#)
 - 3.3.4 [Detection idiom](#)
 - 3.4 [Compile-time rational arithmetic](#)
 - 3.4.1 [Header <experimental/ratio> synopsis](#)
 - 3.5 [Time utilities](#)
 - 3.5.1 [Header <experimental/chrono> synopsis](#)
 - 3.6 [System error support](#)
 - 3.6.1 [Header <experimental/system_error> synopsis](#)
 - 3.7 [Class template propagate_const](#)
 - 3.7.1 [Class template propagate_const general](#)
 - 3.7.2 [Header <experimental/propagate_const> synopsis](#)

- 3.7.3 [propagate_const requirements on T](#)
 - 3.7.3.1 [propagate_const requirements on class type T](#)
 - 3.7.4 [propagate_const constructors](#)
 - 3.7.5 [propagate_const assignment](#)
 - 3.7.6 [propagate_const const observers](#)
 - 3.7.7 [propagate_const non-const observers](#)
 - 3.7.8 [propagate_const modifiers](#)
 - 3.7.9 [propagate_const relational operators](#)
 - 3.7.10 [propagate_const specialized algorithms](#)
 - 3.7.11 [propagate_const underlying pointer access](#)
 - 3.7.12 [propagate_const hash support](#)
 - 3.7.13 [propagate_const comparison function objects](#)
- 4 [Function objects](#)
 - 4.1 [Header <experimental/functional> synopsis](#)
 - 4.2 [Class template function](#)
 - 4.2.1 [function construct/copy/destroy](#)
 - 4.2.2 [function modifiers](#)
 - 4.3 [Searchers](#)
 - 4.3.1 [Class template default_searcher](#)
 - 4.3.1.1 [default_searcher creation functions](#)
 - 4.3.2 [Class template boyer_moore_searcher](#)
 - 4.3.2.1 [boyer_moore_searcher creation functions](#)
 - 4.3.3 [Class template boyer_moore_horspool_searcher](#)
 - 4.3.3.1 [boyer_moore_horspool_searcher creation functions](#)
 - 4.4 [Function template not_fn](#)
- 5 [Optional objects](#)
 - 5.1 [In general](#)
 - 5.2 [Header <experimental/optional> synopsis](#)
 - 5.3 [optional for object types](#)
 - 5.3.1 [Constructors](#)
 - 5.3.2 [Destructor](#)
 - 5.3.3 [Assignment](#)
 - 5.3.4 [Swap](#)
 - 5.3.5 [Observers](#)
 - 5.4 [In-place construction](#)
 - 5.5 [No-value state indicator](#)
 - 5.6 [Class bad_optional_access](#)
 - 5.7 [Relational operators](#)
 - 5.8 [Comparison with nullopt](#)
 - 5.9 [Comparison with T](#)
 - 5.10 [Specialized algorithms](#)
 - 5.11 [Hash support](#)
- 6 [Class any](#)
 - 6.1 [Header <experimental/any> synopsis](#)
 - 6.2 [Class bad_any_cast](#)
 - 6.3 [Class any](#)
 - 6.3.1 [any construct/destruct](#)
 - 6.3.2 [any assignments](#)
 - 6.3.3 [any modifiers](#)
 - 6.3.4 [any observers](#)
 - 6.4 [Non-member functions](#)
- 7 [string view](#)
 - 7.1 [Header <experimental/string_view> synopsis](#)
 - 7.2 [Class template basic_string_view](#)
 - 7.3 [basic_string_view constructors and assignment operators](#)
 - 7.4 [basic_string_view iterator support](#)
 - 7.5 [basic_string_view capacity](#)
 - 7.6 [basic_string_view element access](#)
 - 7.7 [basic_string_view modifiers](#)
 - 7.8 [basic_string_view string operations](#)

- [7.8.1 Searching basic_string_view](#)
 - [7.9 basic_string_view non-member comparison functions](#)
 - [7.10 Inserters and extractors](#)
 - [7.11 Hash support](#)
- [8 Memory](#)
 - [8.1 Header <experimental/memory> synopsis](#)
 - [8.2 Shared-ownership pointers](#)
 - [8.2.1 Class template shared_ptr](#)
 - [8.2.1.1 shared_ptr constructors](#)
 - [8.2.1.2 shared_ptr observers](#)
 - [8.2.1.3 shared_ptr casts](#)
 - [8.2.1.4 shared_ptr hash support](#)
 - [8.2.2 Class template weak_ptr](#)
 - [8.2.2.1 weak_ptr constructors](#)
 - [8.3 Type-erased allocator](#)
 - [8.4 Header <experimental/memory_resource> synopsis](#)
 - [8.5 Class memory_resource](#)
 - [8.5.1 Class memory_resource overview](#)
 - [8.5.2 memory_resource public member functions](#)
 - [8.5.3 memory_resource protected virtual member functions](#)
 - [8.5.4 memory_resource equality](#)
 - [8.6 Class template polymorphic_allocator](#)
 - [8.6.1 Class template polymorphic_allocator overview](#)
 - [8.6.2 polymorphic_allocator constructors](#)
 - [8.6.3 polymorphic_allocator member functions](#)
 - [8.6.4 polymorphic_allocator equality](#)
 - [8.7 template alias resource_adaptor](#)
 - [8.7.1 resource_adaptor](#)
 - [8.7.2 resource_adaptor_imp constructors](#)
 - [8.7.3 resource_adaptor_imp member functions](#)
 - [8.8 Access to program-wide memory_resource objects](#)
 - [8.9 Pool resource classes](#)
 - [8.9.1 Classes synchronized_pool_resource and unsynchronized_pool_resource](#)
 - [8.9.2 pool_options data members](#)
 - [8.9.3 pool_resource constructors and destructors](#)
 - [8.9.4 pool_resource members](#)
 - [8.10 Class monotonic_buffer_resource](#)
 - [8.10.1 Class monotonic_buffer_resource overview](#)
 - [8.10.2 monotonic_buffer_resource constructor and destructor](#)
 - [8.10.3 monotonic_buffer_resource members](#)
 - [8.11 Alias templates using polymorphic memory resources](#)
 - [8.11.1 Header <experimental/string> synopsis](#)
 - [8.11.2 Header <experimental/deque> synopsis](#)
 - [8.11.3 Header <experimental/forward_list> synopsis](#)
 - [8.11.4 Header <experimental/list> synopsis](#)
 - [8.11.5 Header <experimental/vector> synopsis](#)
 - [8.11.6 Header <experimental/map> synopsis](#)
 - [8.11.7 Header <experimental/set> synopsis](#)
 - [8.11.8 Header <experimental/unordered_map> synopsis](#)
 - [8.11.9 Header <experimental/unordered_set> synopsis](#)
 - [8.11.10 Header <experimental/regex> synopsis](#)
 - [8.12 Non-owning pointers](#)
 - [8.12.1 Class template observer_ptr overview](#)
 - [8.12.2 observer_ptr constructors](#)
 - [8.12.3 observer_ptr observers](#)
 - [8.12.4 observer_ptr conversions](#)
 - [8.12.5 observer_ptr modifiers](#)
 - [8.12.6 observer_ptr specialized algorithms](#)
 - [8.12.7 observer_ptr hash support](#)
- [9 Containers](#)

- 9.1 [Uniform container erasure](#)
 - 9.1.1 [Header synopsis](#)
 - 9.1.2 [Function template erase_if](#)
 - 9.1.3 [Function template erase](#)
 - 9.2 [Class template array](#)
 - 9.2.1 [Header <experimental/array> synopsis](#)
 - 9.2.2 [Array creation functions](#)
- 10 [Iterators library](#)
 - 10.1 [Header <experimental/iterator> synopsis](#)
 - 10.2 [Class template ostream_joiner](#)
 - 10.2.1 [ostream_joiner constructor](#)
 - 10.2.2 [ostream_joiner operations](#)
 - 10.2.3 [ostream_joiner creation function](#)
- 11 [Futures](#)
 - 11.1 [Header <experimental/future> synopsis](#)
 - 11.2 [Class template promise](#)
 - 11.3 [Class template packaged_task](#)
- 12 [Algorithms library](#)
 - 12.1 [Header <experimental/algorithm> synopsis](#)
 - [12.2 Search](#)
 - [12.3 Sampling](#)
 - 12.4 [Shuffle](#)
- 13 [Numerics library](#)
 - [13.1 Generalized numeric operations](#)
 - [13.1.1 Header <experimental/numeric> synopsis](#)
 - [13.1.2 Greatest common divisor](#)
 - [13.1.3 Least common multiple](#)
 - 13.2 [Random number generation](#)
 - 13.2.1 [Header <experimental/random> synopsis](#)
 - 13.2.2 [Utilities](#)
 - 13.2.2.1 [Function template randint](#)
- 14 [Reflection library](#)
 - 14.1 [Class source_location](#)
 - 14.1.1 [Header <experimental/source_location> synopsis](#)
 - 14.1.2 [source_location creation](#)
 - 14.1.3 [source_location field access](#)

Editor's note: Suggest move clause 8.12 (observer pointer) up the document to be adjacent to its related header synopsis, or move 8.1 down.

5.2 Revise The Front Matter

Note that in addition to the changes below, there may be a necessary application of an ISO template for clauses 1-3, and a subsequent renumbering,

1 General [general]

1.1 Scope [general.scope]

1. This technical specification describes extensions to the C++ Standard Library (1.2). These extensions are classes and functions that are likely to be used widely within a program and/or on the interface boundaries between libraries written by different organizations.
2. This technical specification is non-normative. Some of the library components in this technical specification may be considered for standardization in a future version of C++, but they are not currently part of any C++ standard. Some of the components in this technical specification may never be standardized, and others may be standardized in a substantially changed form.
3. The goal of this technical specification is to build more widespread existing practice for an expanded C++ standard library. It gives advice on extensions to those vendors who wish to provide them.

1.2 Normative references [general.references]

1. The following referenced document is indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.
-ISO/IEC 14882:2014~~7~~, *Programming Language — C++*
2. ISO/IEC 14882:— is herein called the C++ Standard. References to clauses within the C++ Standard are written as "C++14~~7~~ §3.2". The library described in ISO/IEC 14882:— clauses ~~17–30~~~~20–33~~ is herein called the C++ Standard Library.
3. Unless otherwise specified, the whole of the C++ Standard's Library introduction (~~C++14 §17~~C++17 §20) is included into this Technical Specification by reference.

1.3 Namespaces, headers, and modifications to standard classes [general.namespaces]

1. Since the extensions described in this technical specification are experimental and not part of the C++ standard library, they should not be declared directly within namespace std. Unless otherwise specified, all components described in this technical specification either:
 - modify an existing interface in the C++ Standard Library in-place,
 - are declared in a namespace whose name appends `::experimental::fundamentals_v23` to a namespace defined in the C++ Standard Library, such as `std` or `std::chrono`, or
 - are declared in a subnamespace of a namespace described in the previous bullet, whose name is not the same as an existing subnamespace of namespace `std`.

[*Example:* This TS does not define `std::experimental::fundamentals_v23::chronopmr` because the C++ Standard Library defines `std::chronopmr`. ~~This TS does not define `std::pmr::experimental::fundamentals_v2` because the C++ Standard Library does not define `std::pmr`.~~ — end example]
2. Each header described in this technical specification shall import the contents of `std::experimental::fundamentals_v23` into `std::experimental` as if by

```
namespace std {  
    namespace experimental {  
        inline namespace fundamentals_v2 {}  
    }  
}  
  
namespace std::experimental {  
    inline namespace fundamentals_v3 {}  
}
```

Note for the future: It would have been much simpler if the following syntax were permitted, but that will require a separate proposal through EWG and Core, so would bind against C++20 at the earliest:

```
namespace std::experimental::inline fundamentals_v3::pmr {  
    // contents...  
}
```

3. This technical specification also describes some experimental modifications to existing interfaces in the C++ Standard Library. These modifications are described by quoting the affected parts of the standard and using underlining to represent added text and strike-through to represent deleted text.
4. Unless otherwise specified, references to other entities described in this technical specification are assumed to be qualified with `std::experimental::fundamentals_v23::`, and references to entities described in the standard are assumed to be qualified with `std::`.
5. Extensions that are expected to eventually be added to an existing header are provided inside the `<experimental/meow>` header, which shall include the standard contents of `<meow>` as if by

```
#include <meow>
```

6. New headers are also provided in the `<experimental/>` directory, but without such an `#include`.

Table 1 — C++ library headers

<code><experimental/algorithm></code>	<code><experimental/map></code>	<code><experimental/string></code>
---	---------------------------------------	--

<code><experimental/any></code>	<code><experimental/memory></code>	<code><experimental/string_view></code>
<code><experimental/array></code>	<code><experimental/memory_resource></code>	<code><experimental/system_error></code>
<code><experimental/chrono></code>	<code><experimental/optional></code>	<code><experimental/tuple></code>
<code><experimental/deque></code>	<code><experimental/propagate_const></code>	<code><experimental/type_traits></code>
<code><experimental/forward_list></code>	<code><experimental/random></code>	<code><experimental/unordered_map></code>
<code><experimental/functional></code>	<code><experimental/ratio></code>	<code><experimental/unordered_set></code>
<code><experimental/future></code>	<code><experimental/regex></code>	<code><experimental/utility></code>
<code><experimental/iterator></code>	<code><experimental/set></code>	<code><experimental/vector></code>
<code><experimental/list></code>	<code><experimental/source_location></code>	

1.4 Terms and definitions [general.defns]

- For the purposes of this document, the terms and definitions given in the C++ Standard and the following apply.

1.4.1 [general.defns.direct-non-list-init]

direct-non-list initialization

A direct initialization that is not list initialization.

1.5 Future plans (Informative) [general.plans]

- This section describes tentative plans for future versions of this technical specification and plans for moving content into future versions of the C++ Standard.
- The C++ committee intends to release a new version of this technical specification approximately every year, containing the library extensions we hope to add to a near-future version of the C++ Standard. Future versions will define their contents in `std::experimental::fundamentals_v34`, `std::experimental::fundamentals_v45`, etc., with the most recent implemented version inlined into `std::experimental`.
- When an extension defined in this or a future version of this technical specification represents enough existing practice, it will be moved into the next version of the C++ Standard by removing the `experimental::fundamentals_vN` segment of its namespace and by removing the `experimental/` prefix from its header's path.

1.6 Feature-testing recommendations (Informative) [general.feature.test]

- For the sake of improved portability between partial implementations of various C++ standards, WG21 (the ISO technical committee for the C++ programming language) recommends that implementers and programmers follow the guidelines in this section concerning feature-test macros. [*Note*: WG21's SD-6 makes similar recommendations for the C++ Standard itself. — *end note*]
- Implementers who provide a new standard feature should define a macro with the recommended name, in the same circumstances under which the feature is available (for example, taking into account relevant command-line options), to indicate the presence of support for that feature. Implementers should define that macro with the value specified in the most recent version of this technical specification that they have implemented. The recommended macro name is `"__cpp_lib_experimental_"` followed by the string in the "Macro Name Suffix" column.
- Programmers who wish to determine whether a feature is available in an implementation should base that determination on the presence of the header (determined with `__has_include(<header/name>)`) and the state of the macro with the recommended name. (The absence of a tested feature may result in a program with decreased functionality, or the relevant functionality may be provided in a different way. A program that strictly depends on support for a feature can just try to use the feature unconditionally; presumably, on an implementation lacking necessary support, translation will fail.)

Table 2 - Significant features in this technical specification

Doc. No.	Title	Primary Section	Macro Name Suffix	Value	Header
N3915	<code>apply()</code> call a function with	3.2.2	<code>apply</code>	201402	<code><experimental/tuple></code>

	arguments from a tuple				
N3932	Variable Templates For Type Traits	3.3.1	type_trait_variable_templates	201402	<experimental/type_traits>
N3866	Invocation type traits	3.3.2	invocation_type	201406	<experimental/type_traits>
P0013R1	Logical Operator Type Traits	3.3.3	logical_traits	201511	<experimental/type_traits>
N4502	The C++ Detection Idiom	3.3.4	detect	201505	<experimental/type_traits>
N4388	A Proposal to Add a Const-Propagating Wrapper to the Standard Library	3.7	propagate_const	201505	<experimental/propagate_const>
N3916	Type-erased allocator for std::function	4.2	function_erased_allocator	201406	<experimental/functional>
N3905	Extending std::search to use Additional Searching Algorithms	4.3	boyer_moore_searching	201411	<experimental/functional>
N4076	A proposal to add a generalized callable negator	4.4	not_fn	201406	<experimental/functional>
N3672, N3793	A utility class to represent optional objects	5	optional	201411	<experimental/optional>
N3804	Any Library Proposal	6	any	201411	<experimental/any>
N3921	string_view: a non-owning reference to a string	7	string_view	201411	<experimental/string_view>
N3920	Extending shared_ptr to Support Arrays	8.2	shared_ptr_arrays	201406	<experimental/memory>
N3916	Polymorphic Memory Resources	8.4	memory_resources	201402	<experimental/memory_resource>
N4282	The World's Dumbest Smart Pointer	8.12	observer_ptr	201411	<experimental/memory>
N4273	Uniform Container Erasure	9.1	erase_if	201411	<experimental/vector>, <experimental/deque>, <experimental/forward_list>, <experimental/list>, <experimental/map>, <experimental/set>, <experimental/unordered_map>, <experimental/unordered_set>,
N4391	make_array	9.2.2	make_array	201505	<experimental/array>
N4257	Delimited iterators	10.2	ostream_joiner	201411	<experimental/iterator>

N3916	Type-erased allocator for <code>std::promise</code>	11.2	<code>promise_erased_allocator</code>	201406	<experimental/future>
N3916	Type-erased allocator for <code>std::packaged_task</code>	11.3	<code>packaged_task_erased_allocator</code>	201406	<experimental/future>
N3925	A sample Proposal	12.3	<code>sample</code>	201402	<experimental/algorithm>
N4061	Greatest Common Divisor and Least Common Multiple	13.1.2, 13.1.3	<code>gcd_lcm</code>	201411	<experimental/numeric>
N4531	<code>std::rand</code> replacement	13.2.2.1	<code>randint</code>	201511	<experimental/random>
N4519	Source-Code Information Capture	14.1	<code>source_location</code>	201505	<experimental/source_location>

5.3 Global Search and Replace

There are a couple of repeating patterns in the normative text following the header synopses that should be applied universally. First, replace all opening/closing namespaces matching the following pattern:

```
namespace std {
namespace std::experimental {
inline namespace fundamentals_v23 {

    // some class definition or other specification ...

} // namespace fundamentals_v23
} // namespace std::experimental
} // namespace std
```

An example of updating a header synopsis:

14.1.1 Header <experimental/source_location> synopsis [reflection.src_loc.synop]

```
namespace std {
namespace std::experimental {
inline namespace fundamentals_v23 {

    struct source_location {
        // 14.1.2, source_location creation
        static constexpr source_location current() noexcept;

        constexpr source_location() noexcept;

        // 14.1.3, source_location field access
        constexpr uint_least32_t line() const noexcept;
        constexpr uint_least32_t column() const noexcept;
        constexpr const char* file_name() const noexcept;
        constexpr const char* function_name() const noexcept;
    };

} // namespace fundamentals_v23
} // namespace std::experimental
} // namespace std
```

1. [Note: The intent of `source_location` is to have a small size and efficient copying. — end note]

5.4 Fix Cross-references

Some parts of the TS update wording in the main standard, so require normative updates to the cross-reference immediates.

2.1 Uses-allocator construction [mods.allocator.uses]

20.7.23.10.7 uses_allocator [allocator.uses]

20.7.23.10.7.1 uses_allocator trait [allocator.uses.trait]

20.7.23.10.7.2 uses-allocator construction [allocator.uses.construction]

Next, the remaining section on type-erased allocators should be using the pmr facility from the main `std::pmr` namespace that landed in C++17, as there is no further experimental version of this feature.

8.3 Type-erased allocator [memory.type.erased.allocator]

1. A *type-erased allocator* is an allocator or memory resource, `alloc`, used to allocate internal data structures for an object `x` of type `C`, but where `C` is not dependent on the type of `alloc`. Once `alloc` has been supplied to `x` (typically as a constructor argument), `alloc` can be retrieved from `x` only as a pointer `rptr` of static type `std::experimental::pmr::memory_resource*` (8.5 C++17 §23.12.2 [mem.res.class]). The process by which `rprior` is computed from `alloc` depends on the type of `alloc` as described in Table 15:

Table 15 - Computed `memory_resource` for type-erased allocator

If the type of <code>alloc</code> is	then the value of <code>rprior</code> is
non-existent — no <code>alloc</code> specified	The value of <code>experimental::pmr::get_default_resource()</code> at the time of construction.
<code>nullptr_t</code>	The value of <code>experimental::pmr::get_default_resource()</code> at the time of construction.
a pointer type convertible to <code>pmr::memory_resource*</code>	<code>static_cast<experimental::pmr::memory_resource*>(alloc)</code>
<code>pmr::polymorphic_allocator<U></code>	<code>alloc.resource()</code>
any other type meeting the Allocator requirements (C++14 §17.6.3.5 C++17 §20.5.3.5 [allocator.requirements])	a pointer to a value of type <code>experimental::pmr::resource_adaptor<A></code> where <code>A</code> is the type of <code>alloc</code> . <code>rprior</code> remains valid only for the lifetime of <code>x</code> .
None of the above	The program is ill-formed.

2. Additionally, class `C` shall meet the following requirements:
 - `C::allocator_type` shall be identical to `std::experimental::erased_type`.
 - `x.get_memory_resource()` returns `rprior`.

Then, there are many references to the C++14 standard, denoted thusly: (C++14 §17.6.3.5). They should be replaced as an editorial action with their corresponding reference to the C++17 standard, as (C++17 §20.5.3.5).

Finally, there are a few stylistic cleanups to apply

- Replace all typedefs with `using`
- `propagate_const` and `source_location` should move class definitions out of header synopsis
- Reorder clause 8 so `<memory>` contents are contiguous

6 Acknowledgements

Thanks to the initial reviewers of R0 of this document, that helped produce this simplified document, and especially to Geoffrey Romer as editor of the Fundamentals TS who agreed that much of the fine detail was better left as an editorial task he would have to pick up.

7 References

- [N4617](#) Programming Languages -- C++ Extensions for Library Fundamentals, Version 2 DTS, Geoffrey Romer
- [P0220R1](#) Adopt Library Fundamentals V1 TS Components for C++17, Beman Dawes
- [P0295R0](#) Adopt Selected Library Fundamentals V2 Components for C++17, Walter Brown